Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Gyna Mayerly Yupanqui Romero
Asignatura	Simulación
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Tipo	Grupal
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	Jose O. Guaman Q.
Fecha	22 de Octubre de 2026 23 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado	3

2. Objetivo(s) de la Práctica:

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación.

- Implementar un modelo sencillo en diferentes herramientas.
- Comparar sintaxis, facilidad de implementación y visualización.
- Identificar el ámbito de aplicación de cada herramienta

3. Materiales y reactivos:

- COmputador
- Python
- NetLogo

- Java/JDK

4. Equipos y herramientas

- SimPy.
- NumPy
- Matplotlib.
- NetLogo.
- JDK.
- Sistema operativo Linux o macOS

5. Procedimiento / Metodología

- Se analizó el modelo para entender cómo se diferencian la versión matemática y la versión por simulación.
- Se identificaron las variables y se establecieron las fórmulas: población ($N = 100$), infectados iniciales (5), susceptibles iniciales (95), tasa de infección ($\beta = 0.30$), tasa de recuperación ($\gamma = 0.10$) y tiempo total (100 pasos). Las ecuaciones fueron $dS/dt = -\beta \cdot S \cdot I/N$, $dI/dt = \beta \cdot S \cdot I/N - \gamma \cdot I$ y $dR/dt = \gamma \cdot I$, con $N = S + I + R$. El número reproductivo básico fue $R_0 = \beta/\gamma = 3$.
- Se definió una regla de contagio común para todas las herramientas en cada paso, cada infectado elige al azar a otra persona y, si esta es susceptible y un número aleatorio es menor que β, se contagia. Luego el infectado se recupera si otro número aleatorio es menor que γ. Los cambios se aplican a la vez, usando una copia del estado anterior.
- Se implementó el código en Python, con SimPy, NumPy y Matplotlib, usando la arquitectura Modelo-Vista-Controlador y con apoyo de una herramienta de inteligencia artificial. El modelo contiene la simulación y las ecuaciones, la vista genera las gráficas y el controlador coordina la ejecución. Los resultados se guardaron en un archivo CSV con las columnas tiempo, susceptibles, infectados y recuperados.
- Se implementó el mismo modelo en NetLogo, donde cada persona es un agente con su propio estado (susceptible, infectado o recuperado) además, el modelo importa el CSV de Python para dibujar las curvas de las dos herramientas en los mismos gráficos.
- Se resolvieron las ecuaciones diferenciales con los métodos de Euler y Runge-Kutta de orden 4, para tener una referencia determinista con la que comparar la simulación de Python.
- Se repitió la simulación de Python 30 veces con semillas distintas (0 a 29), para observar cuánto varían los resultados por efecto del azar y calcular el promedio y la banda del 95 %.
- Finalmente, se generaron las gráficas de la comparación entre Python y NetLogo, del modelo SIR en Python, de la simulación frente a la referencia determinista y de los infectados en las 30 réplicas, y se interpretaron los resultados.

6. Resultados esperados

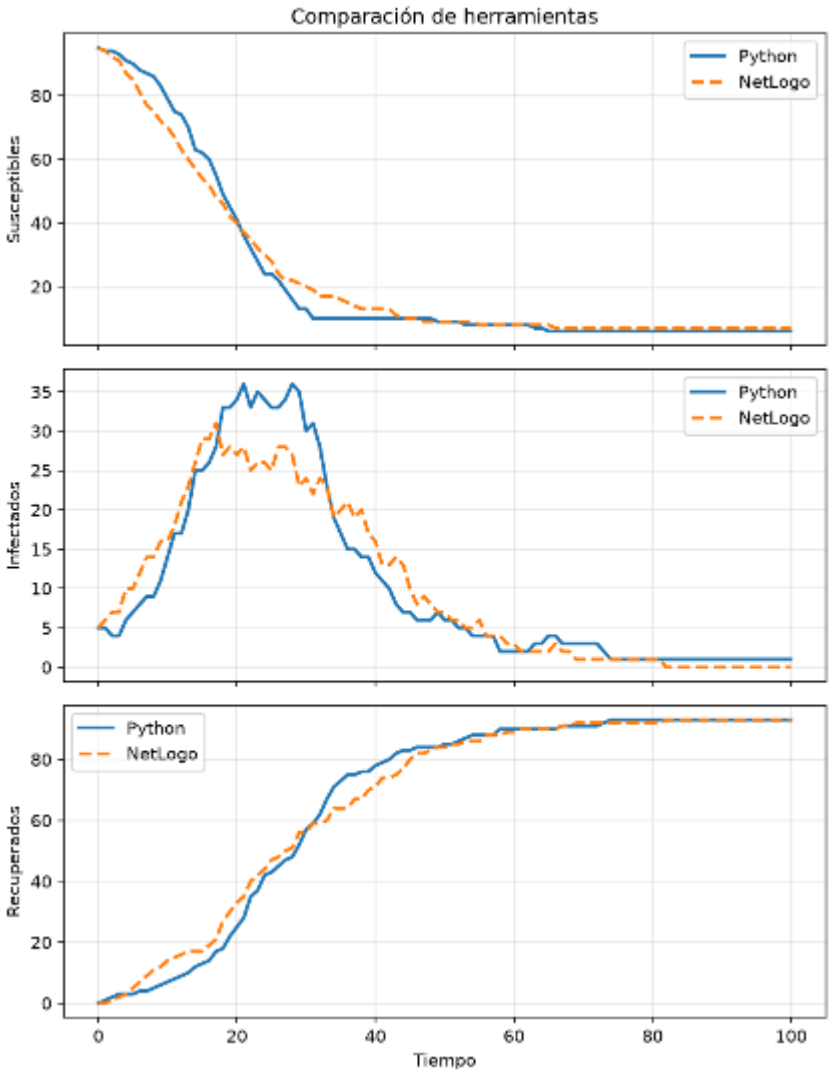


Fig 1. *Comparación de herramientas con los resultado tres casos*

- **Interpretación:**

Se comparan las curvas de Python y NetLogo para susceptibles, infectados y recuperados y se observa que ambas herramientas siguen el mismo comportamiento: los susceptibles disminuyen, los infectados aumentan hasta alcanzar un pico y luego descienden, y los recuperados crecen hasta estabilizarse. Con la semilla 42, las dos terminan con 93 recuperados y entre 6 y 7 susceptibles. Las diferencias se aprecian en el pico de infectados, que en Python fue mayor (36 en t = 21) que en NetLogo (31 en t = 17). Además, en NetLogo los infectados llegaron a 0 cerca de t = 82, mientras que en Python quedó 1 infectado hasta el final. Esta variación se da porque cada herramienta genera su número aleatorio por eso sin importar que sean los mismo parámetros y semilla ocurren contagios en momentos distintos.

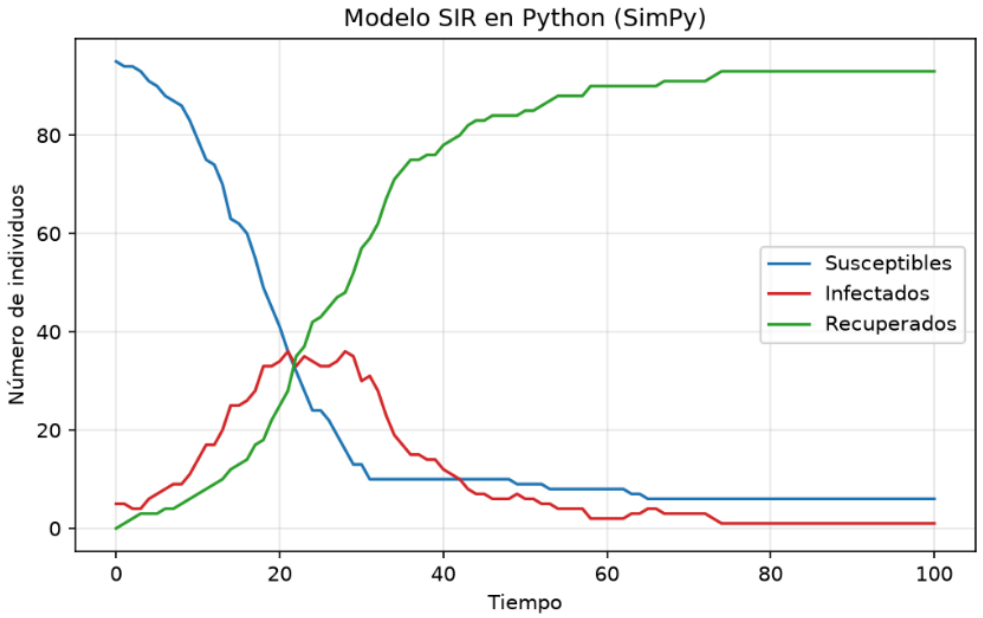


Fig 2. *Modelo SIR con Python*

- **Interpretación:**

En esta se observa la evolución del modelo SIR implementado en Python con SimPy, al inicio hay 95 susceptibles y 5 infectados y a medida que los infectados contagian a otras personas, los susceptibles disminuyen rápidamente hasta cerca de t = 30. Los infectados alcanzan su máximo alrededor de t = 21 y después descienden porque cada vez quedan menos personas que contagiar y los infectados se van recuperando. Los recuperados aumentan de forma constante hasta estabilizarse. Al final de la simulación, 93 de las 100 personas se recuperaron y solo 6 permanecieron sin infectarse, lo que concuerda con el comportamiento esperado de una epidemia con $R_0 = 3$.

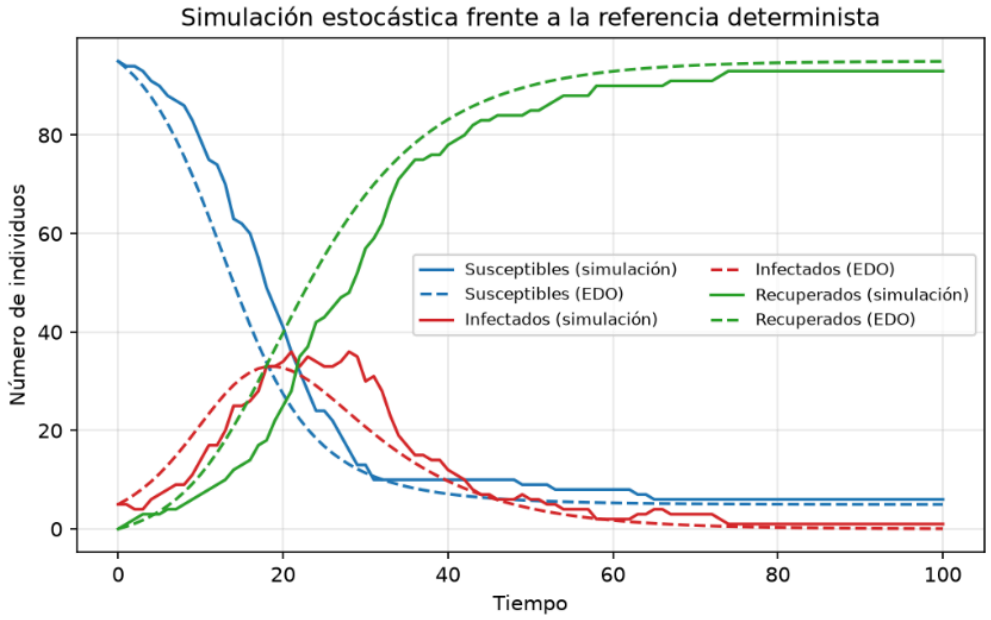


Fig 3. *Simulación estocástica frente a deterministica*

- **Interpretación:**

En esta se compara la simulación estocástica de Python con la solución de las ecuaciones diferenciales del modelo SIR, ambas siguen la misma tendencia general y terminan en valores similares, lo que indica que la simulación está bien implementada. La diferencia es que la solución de las ecuaciones es una curva suave, mientras que la simulación presenta irregularidades y un pequeño desfase en el pico de infectados debido a su aleatoriedad.

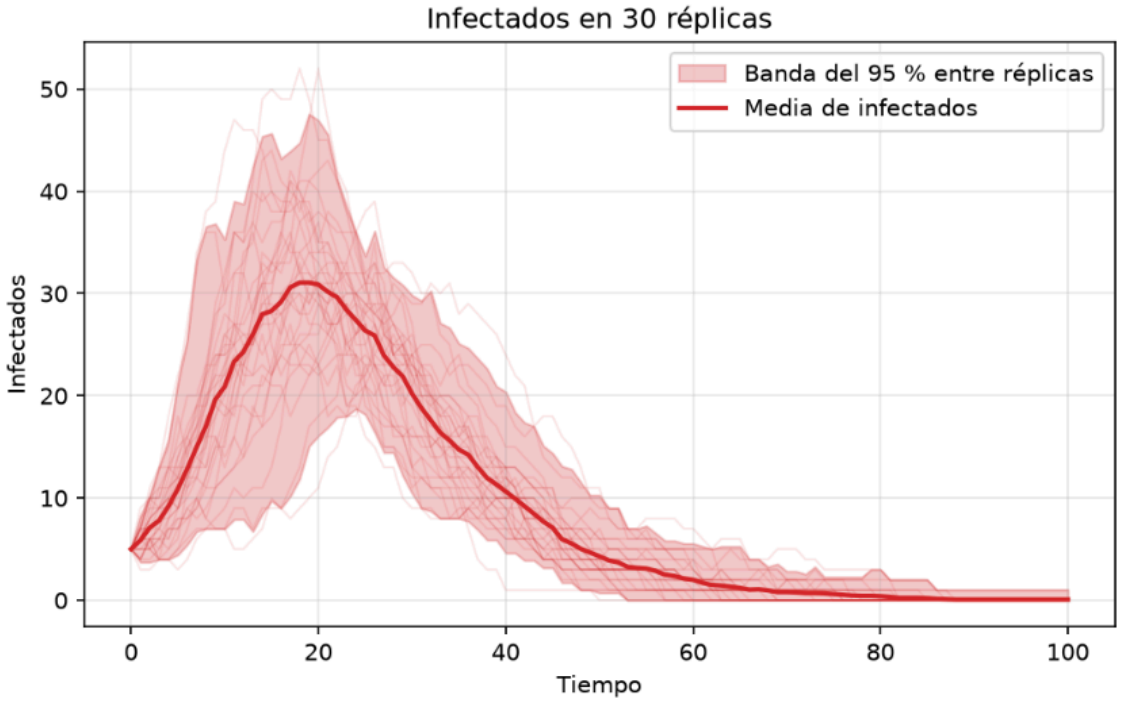


Fig 4. *Infectados en 30 réplicas*

- **Interpretación:**

La Fig. 4 presenta los infectados de 30 réplicas de Python con semillas distintas, junto con su promedio y la banda del 95 %. Todas las réplicas siguen el patrón de subida y bajada, pero el pico cambia en altura y en momento de aparición y la banda es amplia. El pico promedio fue de 36,1 infectados (desviación de 7,0) en t ≈ 19,3, y en promedio terminaron 93,3 recuperados en donde la curva promedio llega a un valor menor (unos 31) porque al promediar réplicas con picos en momentos distintos la curva se suaviza, lo que demuestra que una sola corrida no es suficiente para sacar conclusiones y que el resultado de NetLogo (pico de 31) está dentro del rango de variación normal del modelo.

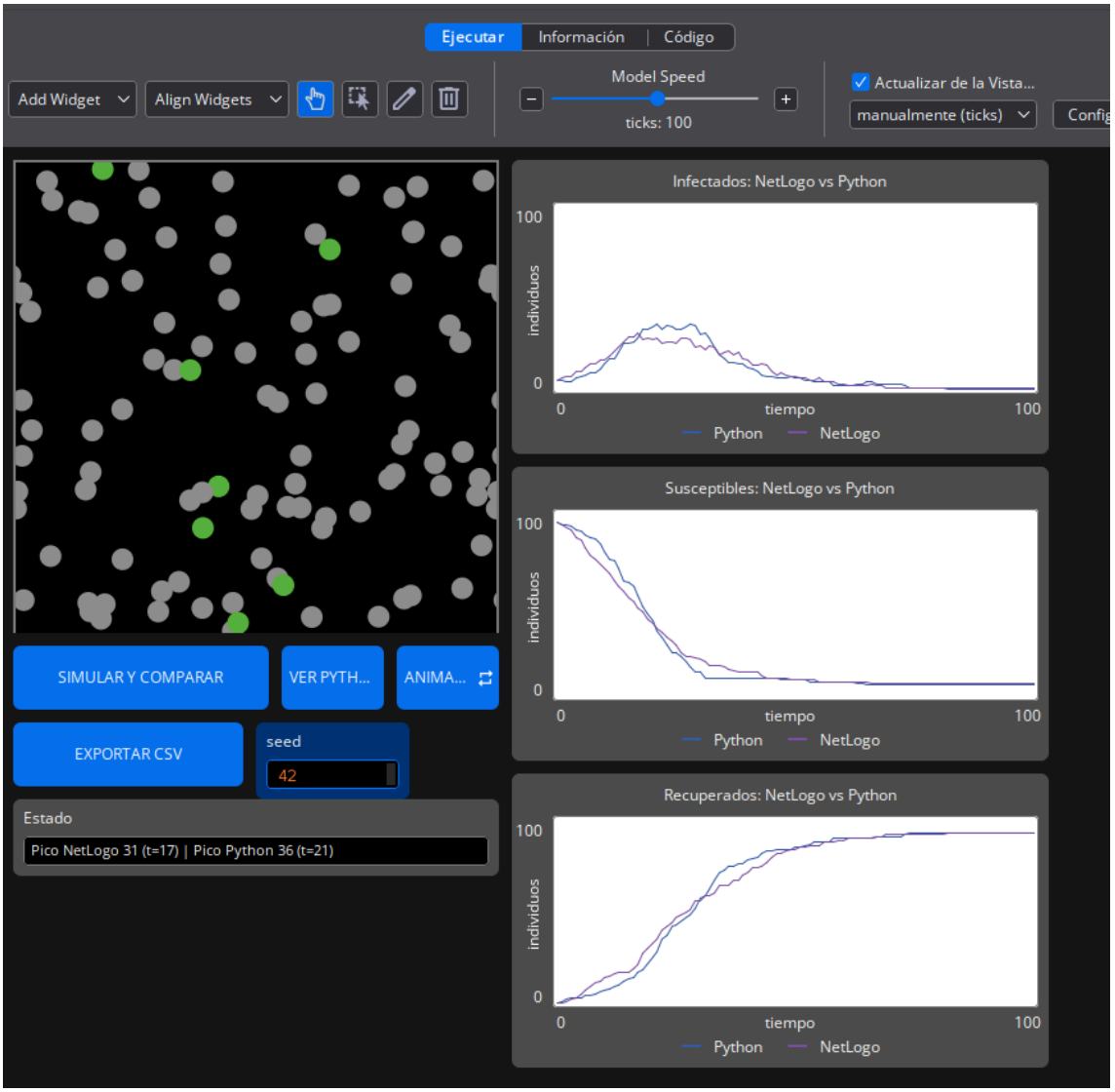


Fig 5. *Implementación de NetLogo*

Análisis de las dos herramientas

Con amables herramientas se hizo el mismo modelo SIR, pero cada una aporta algo distinto.

- **NetLogo** es la más fácil de usar para empezar: ya incluye agentes, botones y un mundo visual donde cada persona es una tortuga que cambia de color (verde, rojo o gris) según su estado, lo que ayuda a entender y a explicar cómo avanza la epidemia.
- **Python** en cambio destaca por su capacidad para generar y analizar datos: con NumPy y Matplotlib se pudieron repetir 30 réplicas, calcular promedios y bandas del 95 %, comparar con las ecuaciones diferenciales y guardar los resultados en un CSV.

De cierta manera ambas herramientas se complementan bien Python para el análisis de datos y NetLogo para realizar vistas animadas. En cuanto a la variedad de resultado de ambas esto se debe por la aleatoriedad que existe cada contagio

y cada recuperación se decide con un número al azar Python y NetLogo generan esos números con algoritmos distintos, la semilla solo indica el punto de partida de la secuencia, así que la misma semilla produce números diferentes en cada herramienta. Además, cada una elige los contactos y recorre a las personas en un orden distinto, por lo que el mismo número aleatorio termina afectando a personas diferentes.

7. Preguntas de Control

- **¿Qué representa el modelo SIR?**

  Representa cómo se propaga una infección en una población que se divide en tres grupos: susceptibles, infectados y recuperados.

- **¿Qué representan los estados S, I y R?**

  S son las personas sanas que se pueden contagiar. I son las que están infectadas y contagian. R son las que ya se recuperaron. Siempre S + I + R = N = 100.

- **¿Qué función cumplen los parámetros β y γ?**

  β es la probabilidad de contagio. A mayor β, la infección se expande más rápido. γ es la probabilidad de recuperación. A mayor γ, los infectados se curan antes. Con β = 0.30 y γ = 0.10, R_0 = 3.

- **¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?**

  Para que la comparación sea justa. Si los valores fueran distintos, no se sabría si la diferencia viene de la herramienta o de los parámetros.

- **¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?**

  Por los números aleatorios, cada una usa su propio generador, así que la secuencia cambia aunque la semilla sea la misma, las curvas tienen la misma forma pero los valores exactos varían.

- **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?**

  Cada individuo es un agente con su propio estado, el mundo se ve en pantalla y es fácil crear interfaces y gráficos es ideal para aprender y explorar.

- **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?**

  Maneja el tiempo y los procesos de forma ordenada con `env.timeout`. además se combina con NumPy y Matplotlib, lo que facilita repetir corridas y analizar datos.

- **¿Qué ventajas ofrece Java para implementar modelos de simulación?**

  Es un lenguaje rápido y estructurado, bueno para modelos grandes o de mucho cálculo. Su desventaja es que requiere más código.

- **¿Qué función cumple el archivo CSV en la integración Python/Java → NetLogo?**

Es el puente entre las dos herramientas Python guarda los resultados y NetLogo los lee para graficarlos, sin generarlos él.

- **¿Qué ventajas presenta separar la generación de datos de la visualización?**

  Se puede cambiar una parte sin tocar la otra, reutilizar los mismos datos en distintas herramientas y repetir las gráficas sin volver a simular. Además, ordena el trabajo y facilita las pruebas.

8. Conclusiones

- Al implementar ambas herramientas los resultados fueron muy parecidos con la semilla 42, los dos modelos terminaron con 93 recuperados y 6 o 7 susceptibles. Solo cambiaron el pico (36 en Python y 31 en NetLogo) y el momento en que ocurrió (t = 21 y t = 17).
- Las diferencias se deben al azar y no a un error, cada herramienta usa un generador distinto, y con una población de solo 100 personas esto se nota más, las 30 réplicas mostraron que un mismo modelo da resultados variables, con picos entre valores bajos y cercanos a 50.
- Una sola corrida no basta para sacar conclusiones es conveniente repetir la simulación y usar el promedio y su variación.
- Finalmente al comparar las dos herramientas las dos tienen su fortaleza NetLogo es mejor para construir y visualizar con agentes de forma rápida por otro lado Python es mejor para generar datos, hacer réplicas y analizar resultados.

9. Anexos

- **Declaración de IA**

  En esta práctica utilicé un agente de programación (el Gentleman, Pi) basado en el modelo Claude de Anthropic como herramienta de apoyo para leer la guía y proponer la regla de contacto común, escribir el modelo en SimPy con arquitectura MVC y sus 9 pruebas, construir el modelo de NetLogo (botones, gráficos e importación del CSV, personas animadas), generar las figuras, medir las réplicas y los tiempos, y redactar borradores de este informe.

  Análisis y correcciones

  - Las decisiones de alcance (sin Java), de la regla de contacto, de la semilla 42 y del contenido las tomé yo. Al revisar lo que generó la IA encontré y corregí tres errores:
  - La semilla de NetLogo estaba en 12345; la cambié a 42 para igualarla con la de Python.
  - El modelo de NetLogo se abría como un modelo vacío en la interfaz; se corrigió y lo comprobé abriéndolo yo mismo.
  - Había generado una parte en Java, que descarté porque estaba fuera del alcance de la práctica.

Verifiqué cada cifra ejecutando el código y revisé el texto antes de entregarlo.

Prompts utilizados

La Tabla 1 recoge, en orden y de forma textual, los principales prompts que escribí al agente.

N°	Prompts utilizados
1	crea el repo del ape 4 grupal como se hizo en el ape 3 y ahi explicas como mis compañeros levantan el codigo, readme e indicaciones todo en español y sencillo
2	en java no hay que hacer
3	solo en phyhton si hicimos en phyhton
4	el informe aun no, tenemos que pulir tema codigo. mergealo y pushea, luego indicame como funciona y como analizarlos desde vscode
5	lo de netlogo lo ponemos en el repo personal no en el grupal no mergees ahi en el grupal, mientras en paralelo explicame el codigo

Tabla 1. *Prompts utilizados con el agente de IA, en orden cronológico*

También se utilizó Notebook para despejar las dudas y consulta sobre las preguntas de control.

Prompt:

"Actúa como un profesor y despeja mis dudas, con explicaciones sencillas y ejemplos explícame qué es un modelo matemático y qué representa el modelo SIR (susceptibles, infectados y recuperados); la diferencia entre variable y parámetro, con ejemplos como t, S(t), I(t) y R(t) frente a N, β, γ, los infectados iniciales y el tiempo total de simulación, incluyendo qué es $R_0 = \beta/\gamma$ y qué significa que sea 3; la diferencia entre un modelo determinístico (las ecuaciones diferenciales del SIR) y uno estocástico (la simulación con números aleatorios), y qué papel cumplen la semilla y las réplicas; la diferencia entre un modelo discreto (que avanza por pasos de tiempo) y uno continuo (ecuaciones diferenciales), y para qué sirve el método de Euler para resolver las ecuaciones del SIR y la diferencia entre

un modelo basado en agentes (NetLogo) y uno basado en pasos de tiempo o eventos (SimPy), explicando por qué dos implementaciones del mismo modelo estocástico pueden dar resultados distintos aunque usen los mismos parámetros y la misma semilla. 7Explícame cada punto por separado, con un ejemplo numérico sencillo cuando sea posible (por ejemplo, con población 100, 5 infectados iniciales, $\beta = 0.30$ y $\gamma = 0.10$), y al final hazme unas preguntas cortas para comprobar si entendí."

Link del Git (Portafolio):

https://github.com/gyna-22/PortafolioSimulacion

Link de Video Explicativo:

https://drive.google.com/drive/folders/1ddfJBqQZZ4TTtgQe_uqrnYHk-9ttPwvL?usp=drive_link